

Docker

Integração de Sistemas - Aula #05

Prof. Dr. Carlos Melo

 carlos.melo@upe.br

Confira antes de começar

- `docker --version`
- `docker run hello-world`

Abra o terminal e rode:



```
1 docker run -d --name banco \  
2   -e POSTGRES_PASSWORD=senha123 \  
3   -p 5432:5432 \  
4   postgres:16-alpine
```

Espere baixar e depois execute:

- `docker exec -it banco psql -U postgres`



```
psql (16.4)  
Digite "help" para ajuda.  
  
postgres=#
```

O que não fizemos

- Não baixamos .exe do Postgres, nem precisamos dar dois cliques para instalar algum programa
- Mas ele está aí e basta um \l e \q para sair do Postgres
- E se quisermos remover, também não precisamos de nenhum procedimento refinado para desinstalá-lo, isso basta:
 - `docker stop banco`
 - `docker rm banco`

O que rolou por baixo dos panos

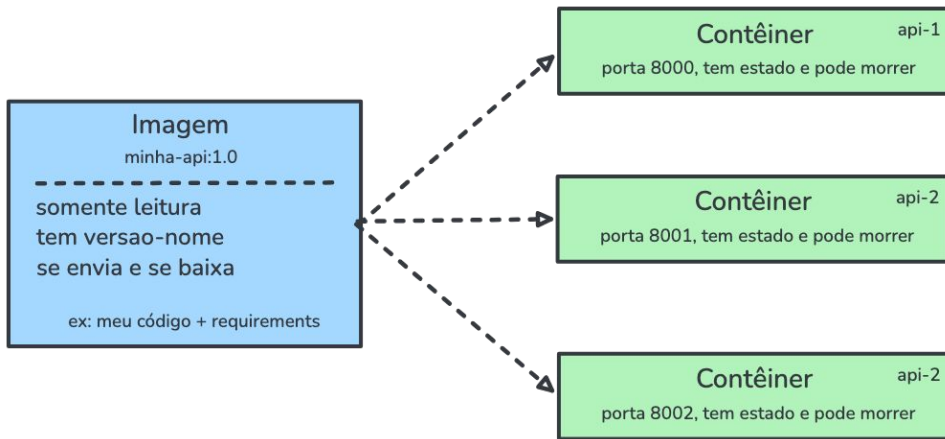
pull: não achou o postgres:16-alpine na máquina e foi buscá-lo no DockerHub

```
> docker run -d --name banco -e POSTGRES_PASSWORD=senha123 -p 5432:5432 postgres:16-alpine
Unable to find image 'postgres:16-alpine' locally
16-alpine: Pulling from library/postgres
5de55e5ef9c0: Pull complete
9441768a3c04: Pull complete
bf5ce8e8fb48: Pull complete
cfa20b885895: Pull complete
262f5476e213: Pull complete
83373364c1a1: Pull complete
e46a888c76da: Pull complete
baa69516db24: Pull complete
e365de836bbc: Pull complete
8bc4580efc9a: Pull complete
0931630a3433: Pull complete
Digest: sha256:cf78e76683b9ca8c5733cbbdce6c9262b45b6767934dd0a95e671f9a0fc20685
Status: Downloaded newer image for postgres:16-alpine
df18b7ddb1731dd17ceb250a93189aed1b9ed3622c206e210ffbb147f200e3e1
```

Create: após baixar postgres:16-alpine ele cria um contêiner com esse "software" instalado.

Imagem vs. Contêiner

- Uma imagem é um molde para um contêiner.
- A partir de uma mesma imagem posso criar quantos contêineres eu quiser.
- Ao apagar o contêiner a imagem continua disponível e ocupando espaço em disco.



Contêiner != VM

- Um contêiner não tem sistema operacional próprio.
 - Ele usa o Kernel do hospedeiro.
- Um contêiner Linux precisa de kernel Linux.
 - No Windows e no Mac, o Docker Desktop roda uma VM Linux escondida para hospedá-los.



Anatomia

```
docker run -d --name banco -e POSTGRES_PASSWORD=senha123 -p 5432:5432 postgres:16-alpine
```

Parte	O que faz	E se faltar?
run	Cria e inicia	-
-d	Roda em background e retorna o ID	Trava no terminal
--name banco	Nome fixo, escolhido por você	Docker inventa um
-e POSTGRES_PASSWORD=	Variável de ambiente dentro do contêiner	Esta imagem não sobe
-p 5432:5432	Porta do host:Porta do contêiner	Nenhuma requisição chega no BD
postgres:16-alpine	imagem:tag	vai buscar o latest

Flags recorrentes

```
docker run --rm -it python:3.12-slim bash
```

- A gente cai dentro de um Linux com Python instalado
 - `--rm` apaga o contêiner assim que ele parar liberar o espaço
 - `-i` mantém a entrada aberta para você digitar
 - `-t` cria a tela do terminal que irá operar o bash
- Teste `python --version` e o que mais quiser antes de fechar por meio do `exit`

Outros comandos úteis

- `docker ps` # os contêineres que estão rodando agora
- `docker ps -a` # todos os contêineres, inclusive os que já morreram
- `docker logs banco` # o que o banco imprimiu em tela enquanto executava
- `docker logs -f --tail 50 banco` # acompanha ao vivo as últimas 50 linhas
- `docker exec -it banco sh` # entra num contêiner que já está de pé
- `docker stop banco` # pede para parar o contêiner do banco
- `docker rm banco` # somente contêineres parados podem ser removidos
- `docker images` # lista as imagens no disco

Primos

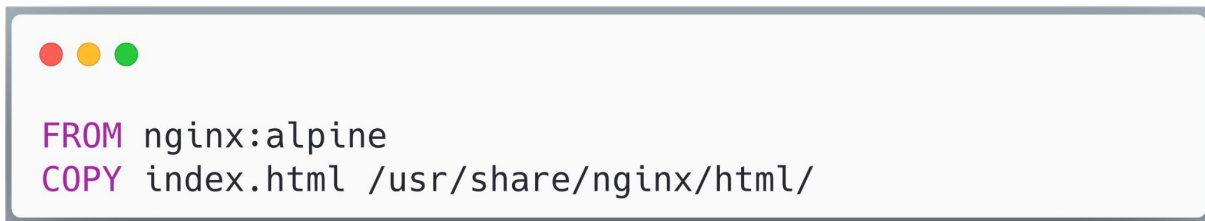
- `docker run` e `docker exec`
 - Um cria um contêiner novo e o outro entra em um que já existe
- `docker stop` e `docker kill`
 - Um envia um SIGTERM e espera 10s para parar o contêiner à força, o que dá tempo para finalizar e salvar operações em andamento. O outro mata na hora.
- `docker rm` e `docker rmi`
 - Um apaga um contêiner específico e o outro uma imagem

Exercício

1. Suba um servidor no ar:
 - a. `docker run -d --name web -p 8080:80 nginx:alpine`
2. Abra no navegador na porta habilitada
3. Veja o contêiner no terminal
4. Veja os logs do contêiner
5. Entre no contêiner em execução
6. Saia do contêiner
7. Pare o contêiner
8. Remova o contêiner
9. Liste as imagens
10. Remova a imagem

Dockerfile

- Geralmente criamos nossas próprias imagens por meio de um arquivo chamado Dockerfile
- Esse arquivo contém o passo a passo para a construção da imagem com uma instrução por linha na ordem em que devem ser executadas

A terminal window with a light gray background and a dark gray border. In the top-left corner, there are three colored circles: red, yellow, and green. The terminal contains two lines of text: 'FROM nginx:alpine' and 'COPY index.html /usr/share/nginx/html/'. The word 'FROM' is in purple, and 'COPY' is in pink.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
```

- Para “executar” esse arquivo e montar a imagem a partir dele usamos o docker build.
- Daí em diante podemos usar todos os comandos vistos anteriormente e operar sobre a imagem personalizada.

Na raiz do seu projeto

- O Dockerfile não tem extensão, mas geralmente está na raiz do projeto com o **D** sempre maiúsculo



- E então é só construir e rodar:
 - `docker build -t meu-site:1.0 .`

Anatomia

```
docker build -t meu-site:1.0 .
```

Parte	O que faz
<code>-t meu-site:1.0</code>	Dá nome:tag a imagem
<code>.</code>	Contexto: Pasta inteira vai para o Docker
<code>-f caminho/Dockerfile</code>	Quando o Dockerfile não está na raiz do contexto
<code>--no-cache</code>	Ignora cache e refaz tudo

```
docker run -d --name site -p 8080:80 meu-site:1.0
```

Expandindo a aplicação

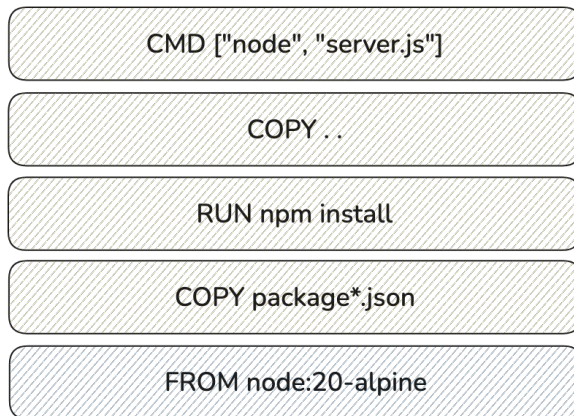


Instrução	O que faz
FROM	Qual a imagem inicial. Sempre na primeira linha.
WORKDIR /app	Cria e entra na pasta, o que vier depois vai rodar a partir de /app
COPY origem destino	Leva arquivos da sua máquina para dentro da imagem
RUN	Executa durante o build, o resultado vira uma camada
EXPOSE 3000	Documenta a porta que a aplicação usa no contêiner
CMD	O comando roda quando o contêiner sobe

Camadas

```
docker history minha-web:1.0
```

- A imagem geralmente é organizada como uma pilha e cada instrução vira uma camada



- **RUN** acontece na construção da imagem
- **CMD** acontece sempre que subimos um contêiner

.dockerignore

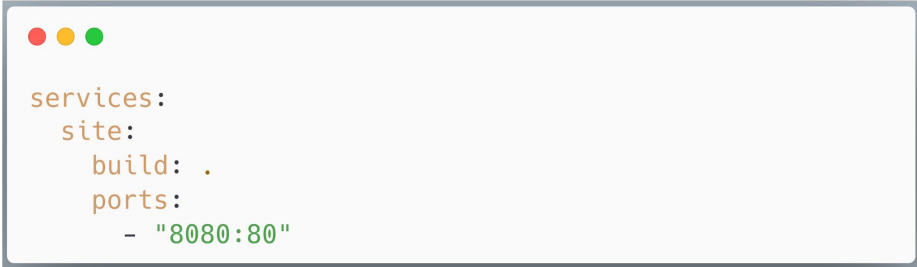
- É usado na raiz do projeto para filtrar o que NÃO queremos copiar para dentro da imagem, em um cenário com:
 - `node_modules`
 - `.git`
 - `.env`
 - `Dockerfile`
 - `*.log`
- O `.dockerignore` impede que determinados arquivos sejam enviados no contexto do build, como um `.env` (segurança) ou o `node_modules` (tamanho e compatibilidade).

Exercício

1. Crie uma pasta chamada meu-site com um Dockerfile e um index.html
2. No index.html, crie uma página simples com seu nome.
3. No Dockerfile, use a imagem nginx:alpine e copie o index.html para /usr/share/nginx/html/
4. Construa a imagem com docker build e confira a imagem docker image ls
5. Execute/crie um novo contêiner em background e dê o nome de site para ele, coloque para rodar na porta 80
6. Acesse http://localhost:8080 e confirme que a página está no ar
7. Altere o index.html e atualize o navegador. O que aconteceu? Por quê?
8. Reconstrua a imagem e crie novamente o contêiner. Confirme que a alteração apareceu.

Docker Compose

Um arquivo de texto que descreve vários contêineres de uma vez: quais são, como cada um sobe, e como eles se ligam um ao outro. Aqui é o ponto de partida para orquestração.

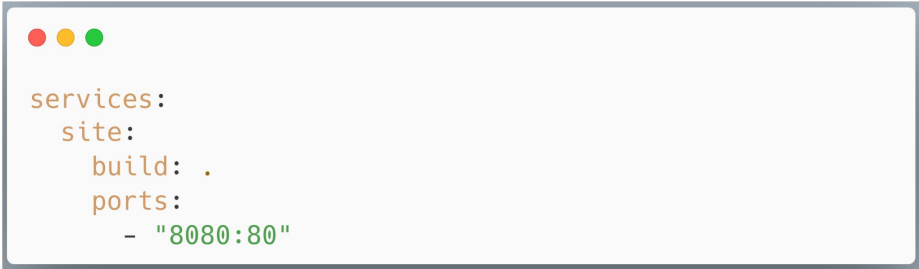


```
services:
  site:
    build: .
    ports:
      - "8080:80"
```

- O arquivo usado pelo docker compose tem a extensão `yaml`.
- O nome padrão adotado é `docker-compose.yaml`

Docker Compose

- Indentação é com espaço e não Tab.
- O nível da hierarquia é com base na quantidade de espaços.
- Elementos em uma lista usam hífen e chave-valor através de dois pontos.
- Aspas nas portas necessárias.



```
services:
  site:
    build: .
    ports:
      - "8080:80"
```

Para executá-lo usamos `docker compose up -d`

Lista de comandos

- `docker compose up -d` # sobe tudo em background, mas não reconstrói se já existe
- `docker compose up -d --build` # reconstrói as imagens antes de subir
- `docker compose ps` # o que está de pé neste projeto
- `docker compose logs -f` # logs de todos os serviços, ao vivo
- `docker compose logs -f api` # logs de um só
- `docker compose exec api sh` # entra num serviço
- `docker compose down` # para e remove contêineres e rede
- `docker compose down -v` # o mesmo, e apaga os volumes

Dois serviços

- O Compose é especialmente útil quando precisamos executar vários serviços juntos.
- Vamos subir nossa aplicação web e um Postgres, juntos, com um comando.
- O que isso obriga a resolver:
 - Como a aplicação encontra o banco
 - Em que ordem eles sobem
 - Onde os dados do banco ficam
 - Como a senha chega até lá

Docker compose

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgres://postgres:senha123@banco:5432/postgres
    depends_on:
      banco:
        condition: service_healthy

banco:
  image: postgres:16-alpine
  environment:
    POSTGRES_PASSWORD: senha123
  volumes:
    - dados:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 3s
    retries: 5

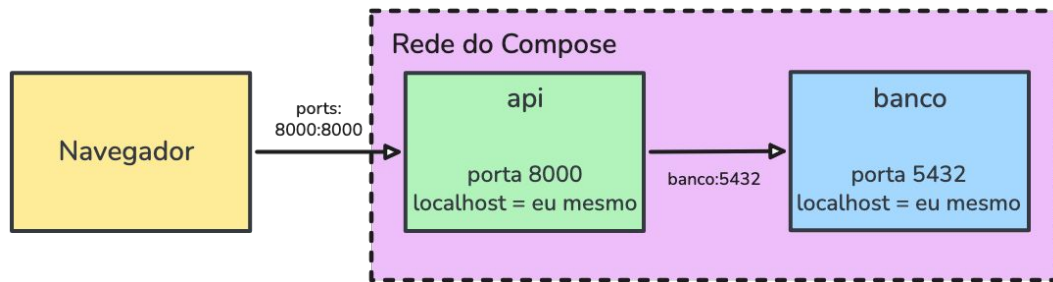
volumes:
  dados:
```

Chave	O que faz
<code>build: .</code>	Constrói a imagem a partir do Dockerfile desta pasta
<code>image:</code>	Usa uma imagem pronta
<code>ports:</code>	Publica para fora, como o <code>-p</code>
<code>environment:</code>	Variáveis dentro do contêiner como <code>-e</code>
<code>volumes:</code>	Onde os dados persistem
<code>depends_on:</code>	Ordem de subida
<code>healthcheck:</code>	Como saber se o serviço está pronto

O banco não precisa expor portas, ele não vai ser acessado externamente, mas sim por nossa aplicação

Como os serviços se encontram?

- Pelo nome.
- Dentro da rede, o nome do serviço é o endereço.
- A API vai se conectar ao banco:5432, sem IP, ports ou informação adicional.



```
docker compose exec api ping banco
```

Volume

- O contêiner é descartável, mas geralmente não queremos perder os dados que ele usa.
- Usamos volumes para aquilo que não queremos perder e precisamos dar nomes aos volumes para que sejam encontrados.

```
    volumes:
      - dados:/var/lib/postgresql/data

volumes:
  dados:
```

depends_on não é o bastante

- Isso garante que o contêiner do banco inicie primeiro.
- A API pode iniciar antes de o PostgreSQL estar pronto para aceitar conexões.
- A api sobe, tenta conectar, recebe `connection refused` e morre.
- Você roda up de novo e funciona, porque agora o banco já está pronto.
- A correção é o `healthcheck`.



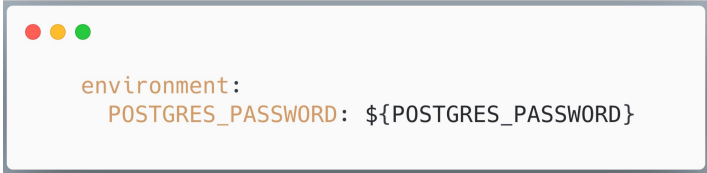
```
depends_on:  
  - banco
```



```
depends_on:  
  banco:  
    condition: service_healthy
```

Segurança

- O `compose.yaml` vai para o repositório e a senha não deve ir junto.
- `.env`, na mesma pasta:
 - `POSTGRES_PASSWORD=senha123`
- `compose.yaml`:

A terminal window with a light gray background and a dark gray border. It has three colored window control buttons (red, yellow, green) in the top-left corner. The text inside is:

```
environment:  
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

```
environment:  
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

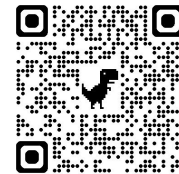
- `.gitignore`
 - `.env`

Exercício para casa

- Execute o contêiner com e sem volumes e veja o comportamento após a inserção de dados em alguma das tabelas. Exemplo sem volume:



```
docker compose up -d
docker compose exec banco psql -U postgres -c "CREATE TABLE teste (id int);"
docker compose exec banco psql -U postgres -c "\dt"      ← a tabela está lá
docker compose down
docker compose up -d
docker compose exec banco psql -U postgres -c "\dt" # "Did not find any relations."
```



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br